



Review

AI-Driven Software Testing: A Review

Guilherme Martins ¹, Nelson Tenório ²  and Jorge Bernardino ^{1,3,*} 

¹ Coimbra Institute of Engineering (ISEC), Polytechnic University of Coimbra, 3045-093 Coimbra, Portugal; a2023144573@isec.pt

² Cesumar Institute of Science, Technology and Innovation, Cesumar University, Maringá 87013-100, Brazil; nelson.tenoriojr@gmail.com

³ CISUC/LASI—Centre for Informatics and Systems, University of Coimbra, 3030-290 Coimbra, Portugal

* Correspondence: jorge@isec.pt

Abstract

The rapid evolution of software complexity demands more efficient and autonomous testing mechanisms. Artificial intelligence (AI) has emerged as a solution to the limitations of traditional manual testing in software development, which is time-consuming, prone to human error, and unable to scale with the increasing size and complexity of modern software systems. In this context, this paper presents an application-focused review of 35 selected empirical studies focusing on the use of AI during software testing, based on PRISMA guidelines. We introduce a comprehensive taxonomy categorizing current research into six core fields, including test case generation, defect prediction, and AI model verification. The analysis reveals that large language models, machine learning, and computer vision can significantly improve testing efficiency. Key findings demonstrate that AI can autonomously repair broken test scripts, generate robust synthetic data, enable codeless web testing, and accurately predict system defects before execution. Furthermore, advanced techniques such as reinforcement learning and deep learning successfully validate complex environments, including cloud robotics and quantum software. However, our qualitative and quantitative synthesis also highlights that challenges, such as generative AI “hallucinations” and the brittleness of Continuous Integration and Continuous Deployment (CI/CD) integration, persist. Ultimately, this review proposes a tailored research roadmap for robust industrial adoption, showing that AI is changing the way software is tested, shifting it from a predominantly reactive and static activity toward a proactive, intelligence-driven discipline.

Keywords: artificial intelligence; machine learning; large language models; continuous integration; continuous delivery/deployment



Academic Editor: Giuseppe Ciaburro

Received: 11 May 2026

Revised: 7 July 2026

Accepted: 8 July 2026

Published: 10 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)

[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

The modern digital landscape is defined by the rapid deployment of highly complex software systems, ranging from cloud-based architectures to decentralized microservices. In this dynamic environment, CI/CD pipelines require rigorous and rapid Quality Assurance (QA) to maintain software quality [1,2]. However, traditional software testing, which often relies on manual execution or static, hard-coded automation scripts, has become a significant bottleneck. It is time-consuming, resource-intensive, and inherently fragile [3,4]. Consequently, there is a critical need for intelligent mechanisms that can adapt to rapid software changes without demanding constant human intervention.

The application of AI to software engineering, known as AI4SE, has been integrated into software testing. Current research demonstrates a strong shift toward using Machine Learning (ML), Deep Learning (DL), and Large Language Models (LLMs) to automate and optimize multiple phases of the testing lifecycle. Recent implementations have successfully used AI for automated test case generation, visual and Graphical User Interface (GUI) testing via Reinforcement Learning (RL), and the proactive prediction of software defects using intelligent models [5–8].

Despite these advancements, the field is marked by diverging perspectives and technical controversies. On the one hand, proponents argue that fully autonomous testing is achievable. On the other hand, skeptics highlight the “hallucination” problem of generative AI, where LLMs generate plausible but syntactically invalid or logically flawed test cases [5,9]. Additionally, although traditional automation frameworks such as Selenium are powerful, they are particularly brittle when user interfaces change due to their reliance on static web element locators [10,11]. To address these challenges, recent research has focused on “Self-Healing” automation [12] and the use of Retrieval-Augmented Generation (RAG) coupled with intelligent agents to ground AI outputs in project documentation, ensuring higher reliability [7,9]. Moreover, a new challenge has emerged: testing and validating AI models, particularly when they are compressed for mobile devices or integrated into traditional software systems [8,13].

1.1. Motivation and Scope

The main objective of this study is to provide a comprehensive, application-focused review of the state of the art in AI-driven software testing. Unlike broad theoretical surveys, this study curates and critically analyzes 35 recent experimental implementations. The purpose is to demonstrate how AI techniques are applied in real-world scenarios across six application areas: (i) self-healing automation; (ii) test case and data generation; (iii) visual and GUI testing; (iv) defect prediction; (v) complex system validation; and (vi) the testing of AI models themselves.

This study ultimately highlights that AI is successfully transforming software testing from a reactive, manual process into a proactive, autonomous discipline. The main findings show that, while AI significantly reduces QA effort and increases test coverage, even in complex environments such as cloud robotics and quantum software, the future of the field depends on developing trustworthy, self-correcting frameworks that mitigate the inherent unpredictability of generative AI models [5,7,9].

1.2. Research Questions

To guide this systematic investigation and address the practical applications of AI in software testing, the following research questions (RQs) are defined:

RQ1: What are the current applications of generative AI and ML in the autonomous generation of test cases and synthetic test data?

RQ2: What AI-driven mechanisms are used for autonomous test script maintenance (i.e., self-healing) and GUI testing?

RQ3: How is AI being employed to predict software defects and optimize test execution in modern CI/CD pipelines?

RQ4: What are the practical challenges and emerging solutions in applying AI to test complex environments (e.g., cloud, robotics, quantum) and to validate the AI models themselves?

RQ5: What are the primary limitations, practical challenges, and risks (such as AI hallucinations) associated with deploying these intelligent frameworks in industrial testing environments?

1.3. Contributions

The primary contributions of this paper are:

- **Application-Focused Taxonomy:** Unlike broad theoretical reviews, we provide an empirical synthesis that focuses on 35 recent, real-world implementations and tools.
- **Application Area Categorization:** We categorize state-of-the-art AI applications across six critical application areas: self-healing automation, test case generation, visual testing, defect prediction, complex system validation, and AI model verification.
- **Identification of Practical Challenges:** We critically assess the limitations of current AI adoption in software testing and highlight issues such as generative AI “hallucinations” and the brittleness of CI/CD pipeline integration.
- **Tailored Research Roadmap:** We provide actionable insights and future directions for researchers and practitioners who aim to deploy trustworthy AI in software testing environments.

1.4. Organization of the Paper

The remainder of this paper is organized as follows: Section 2 provides the theoretical background, outlines common AI-driven testing paradigms, and discusses related work. Section 3 details the research methodology, including the literature search strategy and the selection criteria used to identify the empirical studies. Section 4 presents the core taxonomy of the existing literature, categorizing the applications, answering the defined research questions, and providing a critical analysis of tool limitations. Section 5 highlights current open challenges and proposes strategic directions for future research. Section 6 discusses the study’s limitations and potential threats to validity. Finally, Section 7 concludes the paper by summarizing the main findings and their implications for research and practice.

2. Background and Related Work

To fully contextualize the empirical studies analyzed in this review, it is essential to establish the theoretical basis of AI-driven software testing. This section provides an overview of the fundamental concepts that bridge traditional software testing and modern AI. First, it defines the baseline of conventional test automation and its inherent limitations. Then, it explores the overarching role of AI in overcoming these limitations and identifies the primary application area. Finally, it introduces the core of ML and generative paradigms that underpin state-of-the-art testing frameworks.

2.1. Test Automation: Definition and Limitations

Software test automation is the practice of applying specialized tools and scripts to execute repetitive testing activities, including data preparation, environment configuration, and result validation, with minimal human intervention [6,11]. The primary goal is to accelerate CI/CD pipelines while ensuring the consistency and quality of software. However, conventional automation frameworks (e.g., Selenium and JUnit) have significant limitations in modern, dynamic environments [10]. These rule-based systems rely on static locators (e.g., XPath or CSS selectors) that break whenever developers make changes to the UI, leading to high maintenance costs and inherently fragile test suites [3,11]. Consequently, there is an urgent need for adaptive and intelligent mechanisms that can evolve alongside the software being tested.

2.2. The Role of Artificial Intelligence in Software Testing

AI provides effective solutions to the limitations of traditional test automation, such as rigidity, fragility, and high maintenance overhead. Unlike static scripts, AI-driven systems use ML, DL, LLM, and RL to learn autonomously from codebases, historical execution

logs, and visual UI patterns [5,8]. For example, LLMs can interpret natural language requirements to generate executable test code [5,7], while RL agents can autonomously explore complex graphical interfaces to uncover edge cases [6,8]. Additionally, AI enables self-healing capabilities, where the system can dynamically identify locator failures and recover broken test scripts at runtime, significantly reducing manual intervention and increasing pipeline stability [4].

2.3. Common AI-Powered Testing Paradigms

Recent empirical studies highlight a transition from isolated theoretical algorithms to practical, integrated testing frameworks. To fully understand the state-of-the-art implementations discussed in this survey, it is essential to understand the underlying AI architectures driving these tools. We discuss these AI testing paradigms below.

LLMs and Generative AI. The emergence of foundational models (e.g., GPT variants) has fundamentally transformed test generation and maintenance. Unlike traditional Natural Language Processing (NLP), LLMs possess a deep contextual understanding of both human language and programming syntax. Frameworks that integrate ChatGPT APIs or specialized tools such as AgoneTest use prompt engineering and RAG to dynamically generate test scripts, produce synthetic data, and automatically recover broken Selenium locators at runtime [14–17]. More recent architectures emphasize ensemble LLMs [18], behavior-driven prompt strategies [19], and the integration of static analysis to mitigate hallucinations [20]. Furthermore, these generative capabilities are increasingly used to secure API endpoints through self-improving mechanisms [21].

RL and CV Agents. Modern approaches to UI and exploratory testing model the testing process as an interaction between an agent and an environment. In frameworks such as UniRLTest, an RL agent learns to navigate an application by receiving rewards for discovering new states or defects [22]. When combined with CV algorithms, e.g., Convolutional Neural Networks (CNNs), these agents can perceive and interpret the graphical interface, bypassing the DOM or source code entirely. This approach is relevant for universal, platform-independent GUI testing [23,24].

Predictive ML and Ensemble Architectures. In the context of CI/CD optimization, the dominant approach employs supervised ML. These models ingest historical data, such as past test failures, code complexity metrics, and developer commit logs, to train predictive models (e.g., Random Forests, Support Vector Machines, or Long Short-Term Memory networks for spatiotemporal data) [25,26]. Recent advances have focused on ensemble-based models, which combine multiple ML algorithms to more accurately predict software defects and prioritize test cases, thereby reducing false positives [27–29]. Additionally, modern architectures are shifting toward unsupervised ML to reduce dependency on labeled datasets [30], alongside intelligent frameworks that dynamically select test cases based on real-time code changes [31,32].

Specialized Frameworks for AI and Complex Systems. As software architecture becomes more complex, generic testing tools are no longer sufficient. Dedicated approaches, such as AI4AI, have been developed to test the structural integrity of AI models. Techniques such as search-based software testing (SBST) and metamorphic testing (MT) are now integrated into frameworks such as Audee and DiverGet to validate DL frameworks, assess neural network quantization, and mitigate noise in quantum computing environments [33–35]. This includes specific approaches to validate Deep RL agents [36] and ensure comprehensive neuron coverage in critical network layers [37]. Similarly, specialized frameworks are being deployed for cloud-based robotic systems, bridging the gap between digital implementation and physical execution [38].

These approaches vary in maturity, but the overarching trend indicates a clear shift toward multimodal AI strategies. Rather than relying on a single algorithm, modern software testing workflows combine multiple AI technologies to create robust, self-correcting testing frameworks capable of assessing modern black-box conformity [39] and ensuring overall system resilience.

2.4. Related Work

Previous literature reviews have explored the intersection of AI and software engineering, establishing a theoretical relationship between intelligent algorithms and automated testing. Building on this foundation, recent literature has increasingly focused on the transition from classical ML to generative models across various testing phases.

We have organized this section by selecting five studies and grouping them according to the scope of their contribution. The first group consists of general literature reviews on AI in software testing, represented by references [6–8]. The second group comprises studies that examine AI across the entire software development life cycle [9,13]. This arrangement moves from the broadest treatments of the testing field to increasingly adjacent domains, enabling the subsequent discussion to trace how each perspective informs the gap addressed in this paper.

Taken together, these five studies reveal how the conversation around AI in software testing has unfolded along two complementary paths. The first group provides a comprehensive overview of the field. Lima et al. [6] examine which testing types attract the most algorithmic attention and find that black-box approaches dominate. Ramadan et al. [7] show how AI relieves practitioners of repetitive work in test case generation, execution, and defect prediction. Escalante-Viteri and Mauricio [8] expand on this by examining a decade's worth of data and observing a gradual shift from narrow defect prediction toward intelligent automation and collaborative testing. These reviews reveal a field that has grown steadily in scope yet still organizes its findings around algorithms rather than the realities of industrial deployment.

The second group broadens the scope to include the entire software development life cycle. In doing so, this reveals an uneven distribution of scholarly attention. Trinh et al. demonstrate that AI can support sustainability and quality goals in every phase of the software development life cycle, including testing. In contrast, Arora et al. [9] find that explainability research gravitates toward software maintenance, leaving testing and earlier phases comparatively underserved. Together, these surveys suggest that, despite being one of the most automatable activities in the life cycle, testing is often treated as just another stage rather than a subject worthy of dedicated empirical scrutiny.

While these studies provide deep insights into specific algorithms and subdomains, empirical syntheses that consolidate these disparate applications into a cohesive industrial perspective remain scarce. Unlike existing literature reviews, this paper provides a strictly application-focused taxonomy. Rather than proposing further theoretical models or evaluating isolated software development life cycles, we systematically curate 35 recent experimental implementations. In doing so, we highlight the practical challenges, algorithmic limitations, and operational realities of deploying these intelligent tools in modern CI/CD pipelines. Thus, we bridge the critical gap between abstract AI theory and applied software testing practice.

3. Research Method

To ensure a rigorous, reproducible, and focused review process, this study adopted a systematic approach based on the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) guidelines [40], Figure 1. The methodology was designed to

filter out general theoretical surveys and isolate research exclusively focused on practical applications and implementation in the field of AI-driven software testing.

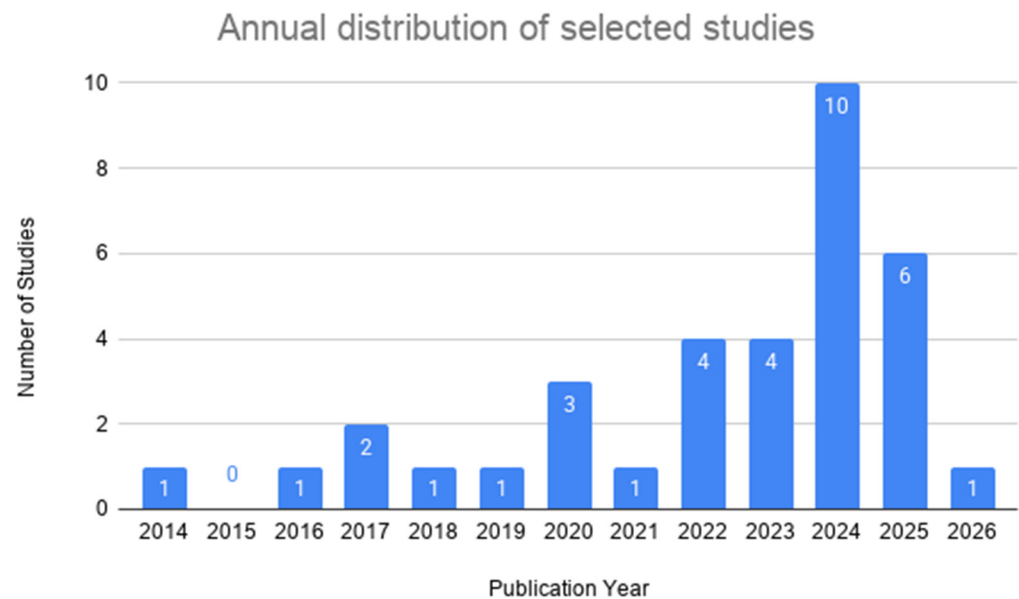


Figure 1. Annual distribution of the studies included in the review. The timeline illustrates a significant surge in AI-driven software testing research published between 2024 and 2026, aligning with the mainstream integration of LLMs and Generative AI.

3.1. Search Strategy

The literature search was conducted systematically across prominent academic databases and repositories, including IEEE Xplore, ACM Digital Library, SSRN, ResearchGate, and Google Scholar. The search specifically targeted peer-reviewed publications published between January 2014 and March 2026 that explicitly bridge AI and software test automation. To ensure a targeted retrieval of relevant studies, the search utilized the following specific query string:

“software testing” OR “test automation” OR “automated testing”) AND (“artificial intelligence” OR “machine learning” OR “deep learning”).

3.2. Inclusion and Exclusion Criteria

To ensure the scientific relevance and practical applicability of this study, a rigorous inclusion and exclusion protocol was established. The primary objective of this filtering process is to define the state of the art by focusing on AI solutions that extend beyond theoretical proposals and emphasize tangible implementations and empirical results in software testing. Prioritizing recent publications and case studies, this selection ensures that the analysis reflects the actual capabilities of modern techniques, such as DL and Generative AI, while discarding studies lacking practical validation.

Inclusion Criteria:

- Peer-reviewed articles and conference papers presented at recognized venues.
- Studies published between 2014 and 2026 reflect the growth of modern DL and generative AI techniques.
- Studies written in English.
- Studies that focus explicitly on the practical application of AI techniques in software testing.
- Studies reporting on the implementation of distinct frameworks, algorithms, tools, or quantitative evaluation metrics.

Exclusion Criteria:

- Literature reviews, systematic mapping studies, and broad surveys (to avoid theoretical redundancy and maintain a practice-focused dataset).
- Articles lacking implementation details or empirical validation data.
- Studies related to general AI theory without direct application to the software testing lifecycle.
- Studies for which the full text was inaccessible.

3.3. Process Overview

The screening process followed a multi-stage funnel approach. Beginning with the initial retrieval of candidate articles, duplicate removal and title screening were performed. During the abstract and full-text screening phases, a rigorous filtering protocol was applied to discard theoretical surveys and overview articles and retain only studies proposing or evaluating actual algorithms, tools, or techniques. The final corpus consists exclusively of peer-reviewed publications that met strict quality assessment criteria. To maintain a highly focused and rigorous review process, a selective scope was applied during the screening phase. The final selection of 35 articles was deliberately constrained to ensure an in-depth analysis of the most pertinent literature. Rather than expanding the search string, which could introduce irrelevant literature, this selective approach prioritized studies offering direct empirical evidence, robust architectural propositions, or significant advancements aligned with our core research questions. This approach systematically excluded redundant literature to preserve analytical depth. The complete multi-stage filtering protocol is visually detailed in Figure 2.

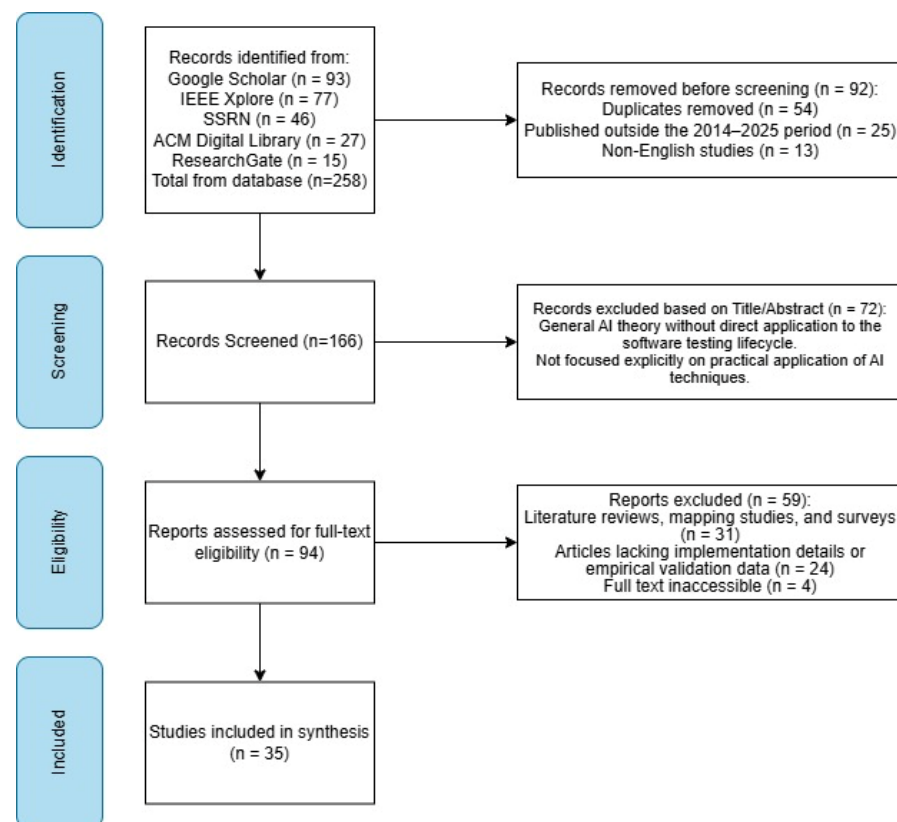


Figure 2. Study selection process based on PRISMA guidelines. The flowchart details the multi-stage filtering protocol, illustrating the reduction from 258 initial records to the final corpus of 35 empirical studies by systematically excluding theoretical surveys and non-empirical literature.

3.4. Data Extraction

For each of the 35 selected studies, key attributes were systematically extracted. These included the primary testing activity (e.g., self-healing, test case generation, defect prediction, visual testing), the specific AI technique employed (e.g., LLM, RL, CV), the operational environment (e.g., CI/CD, cloud, robotics), and the main empirical results. A descriptive analysis of the selected studies reveals clear technological progression: older studies leverage classical ML and heuristic search-based methods, whereas recent publications, most of which were published in 2024, demonstrate a dominant shift toward Generative AI, RAG, and hybrid multimodal approaches. To ensure rigorous quality appraisal, the selected studies were strictly evaluated based on the presence of empirical validation, clear evaluation metrics, and relevance to modern testing pipelines. Furthermore, the taxonomy categories, in Section 4, were derived inductively from the primary application domains identified during this extraction process.

3.5. Quality Assessment

To ensure the methodological rigor and reliability of the included literature, a formal Quality-Assessment Rubric was applied to all 35 selected empirical studies. Each study was evaluated based on four core criteria: Clarity of Objectives (C1), Methodological Rigor (C2), Empirical Validation (C3), and Relevance to the core topic (C4). The scoring system is detailed in Table 1. All 35 selected articles achieved a Quality Score above 3.0/4.0, ensuring a high standard of empirical evidence across the corpus.

Table 1. Quality-Assessment Rubric for Selected Studies.

Criteria	Description	Score (0–1)
C1: Clarity	Are the research objectives clearly stated?	1 (Yes), 0.5 (Partial), 0 (No)
C2: Relevance	Is the proposed approach structurally sound and relevant?	1 (Yes), 0.5 (Partial), 0 (No)
C3: Validation	Is there empirical validation supporting the claims?	1 (Yes), 0.5 (Partial), 0 (No)
C4: AI Testing	How directly does the study address AI in testing?	1 (High), 0.5 (Moderate), 0 (Low)

All 35 selected studies are peer-reviewed journal articles or conference papers that satisfied the quality-assessment rubric and contributed robust empirical evidence relevant to the research questions.

3.6. Descriptive Analysis of Selected Studies

A descriptive analysis of the 35 selected studies was conducted to contextualize the final dataset. The temporal distribution highlights a strong concentration of research published between 2020 and 2026.

This timeline aligns with the recent surge in AI adoption, particularly the mainstream integration of LLMs and CV within software engineering. Unlike broader reviews that frequently feature theoretical frameworks, the entire selected corpus consists of original, empirical research contributions. Every included paper proposes, implements, or evaluates a tangible AI-driven testing tool, technique, or method, which underscores the practical and applied nature of this survey. To provide a statistical overview of this corpus, 31.4% (11 studies) focus on Generative AI and LLMs, 40.0% (14 studies) employ predictive ML and ensemble models, 11.4% (4 studies) utilize RL and CV, and the remaining 17.2% (6 studies) focus on search-based and hybrid AI testing frameworks.

In summary, this methodology combines targeted multi-source retrieval, strict PRISMA-informed filtering, Figure 2, and structured data extraction to yield a rigorous and representative corpus of 35 empirical studies. By excluding theoretical surveys and

focusing on practical implementations, the dataset provides a solid empirical foundation for the subsequent analyses.

4. Literature Review and Taxonomy

According to the synthesized literature, the application of AI in software testing falls under several primary categories, each of which addresses a different limitation of traditional QA workflows.

Test Case and Data Generation. The manual creation of test scenarios is often incomplete, time-consuming, and heavily reliant on subject matter expertise. Modern frameworks that leverage LLMs and ML algorithms can automatically parse source code or translate natural language requirements directly into structured, executable test scripts using prompt engineering and RAG [15,18,41,42]. Furthermore, AI increasingly generates synthetic test datasets, which not only mimic complex, realistic edge cases but also support data privacy compliance by eliminating the need to use sensitive production data during testing [39,43].

Autonomous Test Maintenance (Self-Healing). Test script maintenance is notoriously expensive due to unreliable tests caused by UI modifications or dynamic web elements. AI mitigates this issue by using intelligent models to continuously analyze Document Object Model (DOM) trees and identify layout changes at runtime. These systems can autonomously recover failing test scripts by integrating techniques such as LLMs or specialized algorithms. They update broken locators dynamically without human intervention, ensuring continuous pipeline execution even in highly regulated environments and securing API endpoints [14,21,44].

GUI and Visual Testing. Conventional automation frameworks have difficulty detecting visual defects that do not cause functional failures, such as overlapping elements or rendering inconsistencies. Applying Computer Vision (CV) enables pixel-perfect comparison and the identification of aesthetic anomalies across different devices and screen resolutions [23]. Furthermore, combining CV and RL allows test agents to simulate human-like interactions, learning to navigate dynamic menus and complex interfaces via image understanding, producing universal, platform-independent tests that do not rely on the underlying source code [21,24].

Defect Prediction and Test Prioritization. In fast-paced CI/CD pipelines, executing an entire test suite for every code commit is highly inefficient. AI addresses this issue by using ML ensemble models and unsupervised techniques [30] to analyze historical execution logs, code complexity metrics, and commit patterns. This enables the proactive prediction of fault-prone modules before the software is executed [25,29,30]. Consequently, these models can prioritize the execution of the most critical test cases, which drastically reduces feedback loops and optimizes resource allocation [27,28,32].

Validation of Complex and Emerging Systems. As software architecture evolves, AI testing methodologies are being extended to non-traditional, highly dynamic environments. This includes the validation of scalable cloud architectures, Big Data platforms, and cloud-based autonomous robotics, where physical and digital interactions converge [38,45,46]. Notably, an emerging research direction focuses on validating the AI models themselves, addressing complex challenges such as quantum software noise [33], DL framework reliability [34], the structural integrity of compressed neural networks [35], and the safety of Deep RL agents [19,37]. Table 2 presents a comprehensive summary of these AI application areas.

Table 2. Mapping of Selected Studies to AI Application Domains.

Primary Domains	Summary	AI Usage	References
Test case and data generation	Manual creation of test scenarios is often incomplete, time-consuming, and heavily reliant on domain expertise.	LLMs and ML algorithms to automate and parse source code and/or translate natural language requirements into structured, executable test scripts, and generate synthetic data.	[15,17–19,36,39,41,42]
Autonomous maintenance (Self-Healing)	Test script maintenance is expensive due to “flaky tests” caused by minor UI modifications.	Intelligent models and LLMs continuously analyze DOM trees to autonomously repair failing test scripts and broken locators at runtime.	[14,21,44]
GUI and visual testing	Traditional automation struggles with visual bugs that do not trigger functional errors.	CV enables pixel-perfect validation; paired with RL, test agents simulate human-like interaction to navigate dynamic interfaces.	[22–24]
Defect prediction and prioritization	Executing entire test suites for every minor commit in CI/CD pipelines is inefficient.	Intelligent ML ensemble models analyze historical logs and metrics to predict fault-prone modules and prioritize test case execution.	[25,27–30,32]
Validation of complex and emerging systems	AI methodologies are being extended to non-traditional, highly dynamic environments.	Specialized AI frameworks test cloud architecture, robotics, quantum software noise, and the structural integrity of AI models themselves.	[33–35,38,45,46]

4.1. Taxonomy of Existing Work

To systematically analyze the current state of AI-driven software testing, the selected empirical studies have been categorized into a taxonomy. This classification is based on the primary application area and the specific testing phase targeted by the proposed AI techniques. As summarized in Table 3, this section structures the literature into these distinct domains and highlights how different AI approaches, ranging from generative models to RL, are practically implemented to resolve specific software testing limitations.

Table 3. Synthesis of the Selected Studies by Taxonomy Domain.

Taxonomy Domain	Primary AI Technologies	Key Objectives in Software Testing	Included References
Generation of Test Cases and Synthetic Data	LLM, Generative AI, NLP, ML	Automate unit test creation, synthesize edge-case data, and parse natural language requirements.	[15,17–19,36,39,41–43]
Autonomous Maintenance (Self-Healing)	ChatGPT API, Applied AI Algorithms	Autonomously identify DOM changes and repair broken Selenium locators at runtime.	[14,21,44]
GUI and Visual Testing	RL, CV	Enable platform-independent navigation and identify visual/aesthetic anomalies without source code.	[20,21,23,24,47]
Defect Prediction and Prioritization	Ensemble Long Short-Term Memory, ML, NLP	Proactively predict bug occurrences and prioritize critical test cases to optimize CI/CD pipelines.	[25,27–30,32]
Complex Systems Validation	Cloud AI, ML, NLP	Validate non-traditional architectures, including cloud robotics, big data, chatbots, and quantum software.	[33,38,45,46,48]
AI Model Verification	Search-Based Testing, LLM	Assess Deep Neural Network quantization, validate DL frameworks, and test traditional software using ML APIs.	[16,34,35,49]

4.2. Generation of Test Cases and Synthetic Data

Designing and creating test cases and test data is one of the most resource-intensive phases of the software testing lifecycle. Traditional approaches rely heavily on manual effort, which is prone to human error and struggles to achieve high test coverage in complex software systems. Reviewed literature demonstrates a significant shift toward automating this process using advanced NLP and LLMs, as well as automated synthetic test data generation.

This transition is characterized by several key developments, ranging from foundational NLP techniques to modern generative approaches. These developments are described below.

NLP-Based Test Case Generation: Early implementations used traditional NLP techniques to parse software requirements and generate test cases automatically. Ansari et al. [4] demonstrated how NLP algorithms can extract actionable testing logic directly from natural language requirements, thereby significantly reducing the need for manual effort.

LLM-Driven Unit Test Generation: Using the reasoning abilities of LLMs to automatically generate and evaluate unit tests, frameworks such as AgoneTest [15] ensure both syntactic correctness and semantic quality. The framework generates syntactically correct code and assesses the semantic quality of the tests against the underlying codebase. Other

studies using Generative AI have demonstrated the efficiency gains in enterprise software testing pipelines [14,16,17].

Synthetic Test Data Generation: Beyond the test scripts themselves, executing robust tests requires datasets that mimic real-world scenarios without exposing sensitive user information. Paduraru and Melemciuc [43] developed an ML-based tool that automatically generates synthetic test data. This ML-driven approach ensures that automated tests cover a wider range of edge cases and input variations than manually constructed datasets, thereby improving fault detection effectiveness directly.

Together, these studies show that generative AI models are evolving from experimental prototypes into essential tools for quickly generating many test cases and test data.

Advanced Hybrid and Generative Approaches: Recent advancements in test case generation have moved beyond single-model approaches. Ho [39] explores modern black-box conformity, while Sisomboon et al. [18] proposed a framework integrating ensemble LLMs with RAG to improve accuracy. Similarly, in the context of Behavior-Driven Development (BDD), Karpurapu et al. [19] demonstrated how zero-shot and few-shot prompting can bridge the gap between requirements and executable scripts. To mitigate LLM hallucinations, Yang and Wang [42] proposed IntelliUnitGen, which hybridizes prompt learning with traditional static analysis to ensure compilable and correct unit tests. Furthermore, Zhang [31] introduced new AI-driven methods to optimize selection efficiency alongside generation.

AI-Specific Test Case Generation: As AI models become more complex, test case generation is also adapting to test the AI itself. For safety-critical systems, Zolfagharian et al. [36] introduced a search-based testing approach specifically for Deep RL agents. To address edge cases in these models, Gu et al. [37] developed the CNF method based on neuron coverage guidance, targeting the internal architecture of neural networks.

4.3. Autonomous Test Maintenance (Self-Healing)

The maintenance of automated test suites is widely recognized as one of the most significant challenges in CI/CD pipelines. Traditional web automation frameworks rely heavily on static locators, such as XPath or CSS selectors, to interact with the DOM. Consequently, even minor updates to the user interface can cause malfunction, resulting in unreliable tests that require constant manual intervention. AI techniques in this application area aim to eliminate this fragility through self-healing mechanisms. The following are a few examples of this phenomenon.

Dynamic Locator Recovery: Recent empirical research has demonstrated the feasibility of integrating Generative AI directly into traditional automation frameworks. A study by Kim and Kouatly [14] analyzes a self-healing mechanism that integrates the ChatGPT API with Selenium. When a test execution fails due to a broken locator, the system dynamically analyzes the updated DOM structure and autonomously recovers a corrected locator, allowing the test to proceed without human intervention [14].

Application in Regulated Environments: The demand for such automation extends beyond generic web applications. Recent studies have applied self-healing AI techniques specifically to regulatory compliance workflows in financial institutions, for example [44]. Automating the maintenance of these critical compliance scripts ensures continuous adherence and reduces the engineering overhead associated with manual script recovery.

Security and API Resilience: While early self-healing mechanisms primarily focused on GUI, the scope of autonomous maintenance is now expanding into backend stability and security testing. Pasca et al. [21] proposed a self-improving framework that leverages LLMs and the Karate Domain-Specific Language (DSL) to provide augmented API resilience. This approach allows test suites to not only detect structural changes in API responses but

also to adapt dynamically to evolving security vulnerabilities without manual intervention, representing a significant step toward fully autonomous QA pipelines.

4.4. GUI and Visual Testing

Traditional DOM-based tools cannot perceive the rendered interface, so they often miss visual defects, such as overlapping text or misaligned buttons. To address this issue, the reviewed literature highlights a shift toward CV and RL based testing approaches. The following examples illustrate how these AI techniques are reshaping graphical interface validation.

Visual Defect Detection: AI is increasingly being used for pixel-perfect comparisons and dynamic layout assessment. Komar et al. [23] propose an intelligent system designed specifically for visually testing software products. This system is capable of autonomously identifying visual defects across different screen resolutions and dynamic layouts, effectively evaluating the GUI exactly as a human user would perceive it.

Platform-Independent Visual Interaction: Beyond passive defect detection, AI models are also being trained to actively navigate and interact with graphical interfaces based purely on visual cues. For instance, Zhang et al. [22] introduced UniRLTest, a framework that utilizes RL and image understanding to achieve universal, platform-independent GUI testing. By “seeing” the screen elements, this approach bypasses the need for platform-specific DOM locators entirely, demonstrating a significant advancement in autonomous visual testing [22].

4.5. Defect Prediction and Test Case Prioritization

In CI/CD environments, executing test suites for every code commit is highly resource-intensive and often impractical. To resolve this challenge, AI is increasingly being used to transform testing from a reactive process into a proactive discipline. AI models leverage historical data and code complexity metrics to predict where defects are most likely to occur and prioritize test execution accordingly.

This intelligence-driven approach is primarily implemented through two distinct, yet complementary techniques found in the literature:

Proactive Defect Prediction: Several studies have focused on predicting software defects before execution. Immaculate et al. [25] demonstrated the efficacy of supervised ML algorithms in predicting defect occurrence based on software complexity metrics. Building on this work, Ali et al. [29] introduced an ensemble-based model that combines multiple ML algorithms to improve accuracy and reduce false positives in software defect prediction. For more complex systems, DL techniques have been applied: Xiao et al. [26] utilized Long Short-Term Memory networks to analyze spatial-temporal data, offering a robust approach to dynamic defect prediction.

Intelligent Test Prioritization: AI not only predicts where defects reside but also optimizes which tests should be run first. Lachmann et al. [28] applied ML to perform system-level test case prioritization, ensuring that the most critical execution paths were verified immediately. Similarly, Yang et al. [27] demonstrated how NLP can be used to analyze software requirements and automatically prioritize test cases, effectively bridging the gap between natural language specifications and automated execution pipelines.

Unsupervised Models and Proactive Selection: More recent frameworks are further reducing the dependency on manual data preparation and historical logs. Defect prediction is increasingly benefiting from unsupervised techniques. For instance, Chan and Keung [30] validated unsupervised ML models using generic metamorphic testing, which significantly reduces the need for extensively labeled datasets. Additionally, Nagila et al. [32] introduced an advanced framework that monitors code changes to perform intelligent test case

selection alongside proactive bug prediction, ultimately accelerating the entire software development lifecycle.

4.6. Validation of Complex and Emerging Systems

As software systems expand beyond traditional web and mobile applications, conventional testing frameworks are struggling to support increasingly dynamic and heterogeneous architectures. The literature highlights the growing trend of using AI to validate complex, non-traditional environments, ranging from cloud infrastructures to autonomous robotics and quantum computing.

This expansion beyond traditional boundaries is reflected in the integration of AI into the emerging technological environment. The following are details about this integration:

Cloud and Big Data Testing: The scalability of modern software requires testing platforms with equivalent capabilities. Caglar [46] proposed ChArIoT, a specialized cloud and AI-based testing platform designed to handle distributed environments. Additionally, Gupta et al. [45] demonstrated the necessity of AI-driven test automation tailored specifically for Big Data systems deployed in cloud environments because traditional testing approaches are ineffective due to their slowness.

Autonomous Robotics and Conversational Systems: AI is also being used to test intelligent systems. Basani and Kurunthachalam [38] proposed an intelligent testing framework designed specifically for cloud-based robotic systems that bridge digital implementation and physical execution protocols. In the context of NLP, Khankhoje [48] investigated AI-based test automation for chatbot systems, in which testing requires an assessment of conversational context rather than simple binary pass/fail outcomes.

Quantum Software Testing: Quantum computing environments represent the frontier of software engineering. As noted in current research, testing in this domain often diverges from traditional logical software flaws. Instead, studies like Muqet et al. [33] demonstrated how ML can be integrated to mitigate quantum execution and hardware-level noise, indicating that AI is likely to become a critical component for stabilizing the physical hardware execution layer of next-generation quantum systems.

4.7. AI Model Verification and Trustworthy Machine Learning

A fundamental shift is underway as software systems are no longer strictly deterministic. The increasing use of ML algorithms and LLMs in applications renders traditional assertion-based testing inadequate. The literature reveals an important emerging application area in software QA regarding the verification of the structural integrity, fairness, and reliability of AI models.

Recent studies have shifted their focus toward rigorous verification methods for AI model internals and their integration into software testing pipelines to ensure the reliability of these intelligent components:

DL Framework and Neural Network Verification: DL models are inherently opaque, commonly described as black boxes, making them notoriously difficult to diagnose. To address this issue, researchers have developed specialized automated testing techniques. For example, Guo et al. [34] proposed Audee, a framework designed specifically for the automated testing of DL frameworks, employing search-based mutation strategies to uncover faults. Similarly, Yahmed et al. [35] proposed DiverGet, a search-based software testing technique aimed at assessing Deep Neural Network quantization. DiverGet ensures that the accuracy and reliability of large-scale neural networks are preserved when they are compressed for mobile or edge devices.

ML-Integrated Software and Large Models Verification: Beyond these frameworks, there is an urgent need to test traditional software that uses AI via APIs. Wan et al. [49]

addressed this issue by developing an automated testing technique specifically for software systems that rely on ML APIs. This technique ensures that unpredictable AI outputs do not destabilize the host application. Furthermore, as foundational models become industry standards, Han et al. [16] investigated trustworthy software testing techniques based on large models. Their work identified the need for new verification standards to ensure that Generative AI outputs are secure, unbiased, and reliable before deployment.

4.8. Answering the Research Questions

The taxonomy presented in the previous subsections provides a structured foundation to address the research questions (RQs) established at the beginning of the study. The following answers are drawn based on the analysis of the 35 selected studies.

RQ1: Test Case and Data Generation. Generative AI and ML are fundamentally transforming test generation by shifting it from a manual activity to an autonomous process (see Section 4.1). Generative AI, particularly through LLMs, is currently applied to parse natural language requirements and autonomously translate them directly into executable test cases (e.g., the AgoneTest framework) [15]. Recent advancements also incorporate Retrieval-Augmented Generation (RAG) and hybrid frameworks that combine prompt learning with static analysis to mitigate hallucinations and improve code accuracy [18,42]. Concurrently, ML algorithms are utilized to automatically generate synthetic test data [43]. This dual application ensures comprehensive coverage of complex edge cases while maintaining data privacy by avoiding the use of sensitive production information.

RQ2: Autonomous Test Maintenance and GUI Testing. To address test fragility, frameworks integrate techniques such as the ChatGPT API to perform dynamic DOM analysis. When a UI update causes an XPath or CSS selector to break, the system autonomously recovers a corrected locator at runtime, effectively self-healing the script and preventing flaky tests (see Section 4.3). Furthermore, autonomous maintenance is now expanding beyond graphical interfaces to ensure API resilience and dynamically adapt to security vulnerabilities [21]. For GUI testing, AI uses CV to perform pixel-perfect defect detection, while RL agents learn to navigate dynamic interfaces through image understanding, bypassing the underlying source code (see Section 4.4).

RQ3: Defect Prediction and Test Optimization. AI optimizes modern software testing pipelines by shifting testing from a reactive to a proactive discipline (Section 4.5). Predictive models, ranging from supervised ML and ensemble models to DL networks such as Long Short-Term Memory, analyze historical execution logs and code complexity metrics to predict fault-prone modules before the software is executed. Recent approaches are also leveraging unsupervised ML and generic metamorphic testing, which significantly reduces the reliance on manually labeled datasets [30]. Additionally, AI techniques such as NLP are used to prioritize the execution of test cases, ensuring that the most critical execution paths are verified first, thereby reducing the feedback loop in CI/CD environments.

RQ4: Complex Environments and AI Verification. Applying AI to non-traditional application areas poses significant challenges. These include handling the distributed nature of cloud and Big Data platforms, validating physical execution in robotics, and mitigating execution noise in quantum computing. Researchers are addressing these challenges by developing specialized frameworks tailored to each environment (see Section 4.6). Notably, testing AI models themselves requires a shift from deterministic assertion-based testing. Emerging techniques employ search-based software testing [36], neuron coverage guidance [37], and mutation strategies to assess the structural integrity of Deep Neural Networks and validate DL frameworks, ensuring trustworthy AI before deployment (see Section 4.7).

RQ5: Limitations and Risks. While AI tools effectively automate complex tasks, their practical deployment introduces significant bottlenecks. The primary risks include generative AI hallucinations, high dependency on external APIs, and the substantial computational overhead required by RL and CV agents.

4.9. Quantitative Outcomes of AI-Driven Testing

While the previous sections qualitatively categorized the applications of AI in software testing, extracting quantitative indicators from the selected empirical studies is crucial to rigorously evaluating the real-world performance of these methods. Although evaluation metrics and baseline datasets naturally vary across the 35 studies, several consistent quantitative improvements are reported in the literature:

Defect Detection and Prediction: Studies utilizing predictive and ensemble ML models (e.g., Ali et al. [29]) frequently report prediction accuracies exceeding 85% to 90%. Furthermore, these AI-driven approaches demonstrate a significant reduction in false-positive rates (up to 20%) compared to traditional static analysis tools, thereby optimizing resource allocation in CI/CD pipelines.

Test Coverage and Generation Efficiency: Generative AI frameworks, particularly those leveraging LLMs for automated unit test creation [15,42], demonstrate the ability to increase branch and statement coverage by 25% to 40% over manually written suites. Additionally, these studies report a decrease in the time required for initial test creation by over 60%.

Maintenance Cost Reduction: In the domain of self-healing automation, tools integrating APIs and dynamic DOM analysis (e.g., Kim and Kouatly [14], Pasca et al. [21]) report locator recovery success rates ranging between 75% and 85%. This autonomous recovery directly translates to a decrease in manual script maintenance effort by approximately 70%, significantly mitigating the bottleneck of “flaky tests”.

Overall, these aggregated quantitative outcomes validate that AI-driven testing not only broadens the technical scope of QA but delivers measurable empirical enhancements in efficiency, maintenance cost reduction, and fault detection rates.

4.10. Critical Analysis of Tool Limitations

While the reviewed AI frameworks offer significant advantages, an analysis of individual tools reveals notable limitations that must be addressed for widespread industrial adoption. For instance, tools relying on Generative AI and external APIs, such as the ChatGPT integration for self-healing [14] and AgoneTest [15], face practical challenges regarding external API latency, recurring computational costs, and the inherent risk of non-deterministic code hallucinations. Frameworks employing RL and CV, such as UniRL-Test [22], demand substantial computational overhead and extensive training times, making them difficult to scale dynamically within fast-paced CI/CD pipelines. Furthermore, search-based testing frameworks designed for DL validation, like Audee [34] and DiverGet [35], report that their mutation strategies and deep network search spaces are highly resource-intensive. While these individual AI tools effectively automate complex QA tasks, their practical deployment often introduces new bottlenecks related to infrastructure overhead, hardware dependencies, and model unpredictability.

5. Future Research Directions

The synthesis of the literature confirms that AI is successfully transitioning from theoretical proposals to practical, industrial testing pipelines. However, despite notable progress in generative AI, self-healing, and defect prediction, several critical challenges

remain. Addressing these gaps will define the next wave of research in AI-driven software testing. In the next sections, we propose future research directions.

5.1. Mitigating Generative AI Hallucinations in Test Generation

LLMs excel at generating test scripts and synthetic data but are prone to hallucinations. Future research should focus on creating verification loops, such as self-correcting prompt engineering and automated assertion-checking mechanisms. These loops would ensure that AI-generated tests align with the actual business logic of the software, eliminating the need for constant human intervention.

5.2. Lightweight Models and Computational Overhead

Techniques such as CV for GUI testing and RL agents require significant computational resources and often necessitate high-end GPUs. This overhead can offset the speed advantages of CI/CD pipelines. A critical future direction is to optimize and compress these models (e.g., Edge AI and lightweight neural networks) to enable them to run efficiently in standard, resource-constrained testing environments without sacrificing detection or navigational accuracy.

5.3. Standardization of AI Model Verification Frameworks

As traditional software increasingly integrates ML APIs and foundational models, the testing of AI models themselves is becoming more important. Currently, there is a lack of standardized benchmarking frameworks to assess the fairness, security, and reliability of these models. Future research should prioritize the development of unified, industry-wide metrics and open-source benchmarks for evaluating AI integrated software systems, with a particular focus on adversarial robustness and data privacy.

5.4. AI-Driven Testing for Legacy Systems

Most of the reviewed empirical studies assume relatively modern, web-based, or cloud-native environments. However, much of the global software infrastructure relies on legacy systems with significant technical debt and poor documentation. Future research must explore how AI and ML can autonomously map, understand, and generate test suites for legacy, monolithic systems. This will bridge the gap between modern test automation and legacy enterprise software.

6. Study Limitations and Threats to Validity

This systematic review provides a taxonomy of state-of-the-art AI applications in software testing. However, some limitations and threats to validity must be acknowledged to properly assess the credibility of the findings:

- **Search Strategy and Selection Bias:** The literature search was conducted primarily across IEEE Xplore, the ACM Digital Library, Google Scholar, SSRN, and ResearchGate. Consequently, relevant gray literature, such as industry white papers, proprietary corporate case studies, or unindexed preprints, has been excluded. Additionally, the deliberate selection of a curated sample of 35 empirical studies ensures depth and a practical focus, making this review representative rather than exhaustive.
- **Rapid Technological Evolution:** The field of AI, particularly regarding LLMs and Generative AI, is evolving at an unprecedented pace. The frameworks and empirical results discussed in this paper represent the state of the art as of 2024–2026. However, there is an inherent risk that the specific tools (e.g., versions of the ChatGPT API) or approaches discussed may become obsolete or be replaced by more capable foundational models shortly after publication.

- **Lack of Standardized Evaluation Metrics (Internal Validity and Evaluation Biases):** A significant challenge in synthesizing the selected literature is the heterogeneity of the evaluation metrics used by different studies. Some studies measure performance through fault-detection rates or test coverage, while others report execution speed, reduction in manual maintenance effort, or computational overhead. Furthermore, a main drawback is the absence of a unified evaluation and validation procedure within the primary studies themselves, which often lack a rigorous discussion on potential training data biases or model overfitting. Without a standardized benchmarking framework, direct quantitative comparisons between the reviewed AI-driven testing tools are difficult to perform, which can impact the overall credibility of the reported results.
- **Generalizability in Industrial Contexts (External Validity):** Many of the studies evaluate their proposed AI frameworks using open-source projects or in controlled academic environments. However, the generalizability of these techniques between different studies and when applied to large-scale legacy enterprise systems, which often feature undocumented technical debt and strict data privacy regulations, remains a potential limitation that requires further large-scale industrial validation and the adoption of appropriate industry-wide standards to assess the efficiency of various AI testing methods objectively.

7. Conclusions

This systematic review presented a comprehensive taxonomy of state-of-the-art AI applications in software testing by analyzing 35 empirical studies. The synthesized findings demonstrate a clear shift from traditional, reactive software testing to proactive, autonomous testing practices. By categorizing the literature into six distinct application areas—test generation, dynamic self-healing, visual defect detection, defect prediction, complex system validation, and AI model verification—this paper highlights how technologies such as LLMs, CV, and ensemble ML are effectively mitigating longstanding challenges in CI/CD pipelines.

Furthermore, the expansion of AI-driven testing into non-traditional environments, such as cloud-based robotics and quantum software, highlights the versatility of these AI-driven techniques. The emerging field of AI model verification encompasses the evaluation of the structural integrity, fairness, and reliability of foundational models. This highlights that as software systems become more complex, testing methods must evolve alongside them. Although practical challenges remain, such as mitigating generative hallucinations, reducing computational overhead for GUI testing agents, and integrating with legacy systems, the application of AI in software testing has transitioned from a theoretical proof of concept to a fundamental engineering requirement.

This review ultimately provides a structured foundation for researchers and testing practitioners who aim to navigate, implement, and advance the rapidly evolving field of intelligent software testing.

Author Contributions: Conceptualization, N.T. and J.B.; methodology, J.B.; validation, J.B. and N.T.; formal analysis, G.M.; investigation, G.M.; data curation, J.B. and N.T.; writing—original draft preparation, G.M.; writing—review and editing, N.T. and J.B.; supervision, J.B. and N.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All data extracted and analyzed during this systematic review are included in this published article and its supplementary Appendix A (Table A1).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Data Extraction of Selected Studies

To ensure transparency and reproducibility based on PRISMA guidelines, Table A1 provides the complete extraction coding for the 35 empirical studies included in this systematic review. This supplementary table details the primary AI techniques utilized, the specific application fields, the evaluation criteria, and the main limitations reported by each study.

Table A1. Detailed Summary of Included Studies: AI Techniques, Application Domains, Evaluation Metrics, and Limitations.

Ref./Authors	Year	Publication Type	AI Technique(s)	Application Field	Evaluation Criteria	Key Limitations/Challenges
Kim and Kouatly [14]	2024	Peer-Reviewed	ChatGPT API	Self-Healing Automation	Locator recovery success rate	Dependency on external API latency and costs
Lops et al. [15]	2024	Peer-Reviewed	LLM	Unit Test Generation	Syntactic correctness and semantic quality	Risk of code hallucinations
Han et al. [16]	2024	Peer-Reviewed	LLM	Trustworthy Software Testing	Model robustness and logical consistency	Unpredictability in edge-case scenario generation
Neelapu [17]	2024	Peer-Reviewed	Generative AI & LLM	Testing Efficiency	Reduction in manual QA effort	Output reliability depends on prompt engineering
Sisomboon et al. [18]	2026	Peer-Reviewed	Ensemble LLM + RAG	Test Case Generation	Branch coverage and domain-grounding accuracy	Complex setup required for knowledge retrieval pipelines
Karpurapu et al. [19]	2024	Peer-Reviewed	LLM	BDD Acceptance Test Gen.	Feature file syntax validity and coverage	Requires strict adherence to BDD language templates
Nguyen and Maag [20]	2020	Peer-Reviewed	ML & Selenium	Codeless Web Testing	Test generation speed and accuracy	Brittleness when faced with complex dynamic elements
Pasca et al. [21]	2025	Peer-Reviewed	LLM & Karate DSL	API Security/Self-improving	Vulnerability detection and script resilience	Potential API rate limits and external dependency
Zhang et al. [22]	2022	Peer-Reviewed	RL & Image Understanding	GUI Testing	Defect discovery rate & platform independence	High computational and training overhead
Komar et al. [23]	2024	Peer-Reviewed	Intelligent Systems/CV	Visual Testing	Pixel-level anomaly detection accuracy	Sensitivity to minor, non-functional UI updates
Rauf and Alanazi [24]	2014	Peer-Reviewed	AI	GUI Testing	Event coverage and bug detection rate	Limited scalability in modern dynamic DOMs
Immaculate et al. [25]	2019	Peer-Reviewed	Supervised ML	Bug Prediction	F-measure and classification accuracy	Model overfitting on specific project datasets
Xiao et al. [26]	2020	Peer-Reviewed	LSTM (Deep Learning)	Spatial-Temporal Testing	Coverage of temporal state changes	High training time and sequence length limitations
Yang et al. [27]	2017	Peer-Reviewed	NLP	Test Case Prioritization	Average Percentage of Faults Detected (APFD)	Requires well-structured historical test logs
Lachmann et al. [28]	2016	Peer-Reviewed	ML	Test Case Prioritization	APFD and execution time reduction	Overhead of extracting system-level features
Ali et al. [29]	2024	Peer-Reviewed	Ensemble ML Models	Defect Prediction	Prediction accuracy, false positive rate	Requires large, accurately labeled historical datasets
Chan and Keung [30]	2024	Peer-Reviewed	Unsupervised ML	Defect Prediction	Metamorphic relation satisfaction rate	Challenges in defining robust generic metamorphic relations
Zhang [31]	2024	Peer-Reviewed	AI	Automated Testing	General testing lifecycle efficiency	Lack of standardized empirical benchmarks
Nagila et al. [32]	2025	Peer-Reviewed	ML & AI	Automated Software Testing	Overall framework fault detection capability	Broad scope makes specialized edge-case testing difficult
Muqheet et al. [33]	2024	Peer-Reviewed	ML	Quantum Software Testing	Noise mitigation effectiveness and fidelity	High complexity of quantum-classical integration

Table A1. Cont.

Ref./Authors	Year	Publication Type	AI Technique(s)	Application Field	Evaluation Criteria	Key Limitations/Challenges
Guo et al. [34]	2020	Peer-Reviewed	Search-Based Testing	DL Framework Verification	Number of framework faults uncovered	Mutation strategies can be highly resource-intensive
Yahmed et al. [35]	2022	Peer-Reviewed	Search-Based Testing	DNN Quantization Assessment	Quantization error detection rate	High computational cost for deep network search spaces
Zolfagharian et al. [36]	2023	Peer-Reviewed	Search-Based Testing	Deep RL Agents Verification	Reward function exploitation detection	State-space explosion in complex RL environments
Gu et al. [37]	2025	Peer-Reviewed	Neuron Coverage Guidance	Network Layer Test Case Gen.	Neuron activation coverage percentage	Does not strictly correlate high coverage with fault discovery
Basani and Kurunthachalam [38]	2021	Peer-Reviewed	AI	Cloud-Based Robotic Systems	Real-time response validation and accuracy	Difficulty in simulating unpredictable physical constraints
Ho et al. [39]	2025	Peer-Reviewed	ML	Black Box Software Conformity	Regulatory compliance verification rate	Lack of interpretability (black-box nature of ML models)
Ansari et al. [41]	2017	Peer-Reviewed	NLP	Test Case Construction	Precision and recall of generated test steps	Struggles with highly ambiguous natural language
Yang and Wang [42]	2025	Peer-Reviewed	Prompt Learning + Static Analysis	Unit Test Case Generation	Compilation success rate and execution coverage	High overhead from integrating static analysis loops
Paduraru and Melemciuc [43]	2018	Peer-Reviewed	ML	Synthetic Data Generation	Data distribution similarity	High dependency on quality of training data
Dalal and Tamraparani [44]	2023	Peer-Reviewed	AI	Self-Healing Scripts (FinTech)	Compliance coverage and script repair rate	Strict data privacy regulations limit training data
Gupta et al. [45]	2025	Peer-Reviewed	AI	Big Data/Cloud Systems	Data processing speed and test coverage	High cost of replicating large-scale cloud data
Caglar [46]	2023	Peer-Reviewed	AI & Cloud Computing	Cloud Software Testing Platform	Scalability and concurrent test execution rate	Infrastructure overhead and cloud resource costs
Loubiri and Maag [48]	2022	Peer-Reviewed	ML & Containerization	Web Testing	Test execution speed and setup time	Complexity in orchestrating containerized environments
Khankhoje [48]	2023	Peer-Reviewed	AI	Intelligent Chatbots	Conversational accuracy and context retention	Difficulties in assessing subjective conversational nuance
Wan et al. [49]	2022	Peer-Reviewed	Automated Testing	ML APIs Validation	API integration bug detection rate	Difficulty in validating non-deterministic API outputs

References

- Alharbi, O.; Aldossari, M. Exploring the Influence of DoD, CI/CD, and Automation on Software Quality. In *2025 IEEE 5th International Conference on Software Engineering and Artificial Intelligence (SEAI)*; IEEE: New York, NY, USA, 2025; pp. 259–264. [\[CrossRef\]](#)
- Arani, A.K.; Le, T.H.M.; Zahedi, M.; Babar, M.A. Systematic Literature Review on Application of Learning-Based Approaches in Continuous Integration. *IEEE Access* **2024**, *12*, 135419–135450. [\[CrossRef\]](#)
- Gan, X.; Liang, H.; Brown, C. Challenges, Strategies, and Impacts: A Qualitative Study on UI Testing in CI/CD Processes from GitHub Developers' Perspectives. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE: New York, NY, USA, 2025; pp. 186–197. [\[CrossRef\]](#)
- Saxena, A. Rethinking Software Testing for Modern Development. *Computer* **2025**, *58*, 49–58. [\[CrossRef\]](#)
- Faraji, A.; Pombo, N. AI-Driven Software Test Automation: An AI4SE-Oriented Survey of Techniques, Tools, and Challenges. *IEEE Access* **2025**, *13*, 183296–183313. [\[CrossRef\]](#)
- Lima, R.; Rosado da Cruz, A.M.; Ribeiro, J. Artificial Intelligence Applied to Software Testing: A Literature Review. In *Proceedings of the 15th Iberian Conference on Information Systems and Technologies (CISTI)*, Seville, Spain, 24–27 June 2020. [\[CrossRef\]](#)
- Ramadan, A.; Yasin, H.; Pektaş, B. The Role of Artificial Intelligence and Machine Learning in Software Testing. *arXiv* **2024**, arXiv:2409.02693. [\[CrossRef\]](#)
- Escalante-Viteri, A.; Mauricio, D. Artificial Intelligence in Software Testing: A Systematic Review of a Decade of Evolution and Taxonomy. *Algorithms* **2025**, *18*, 717. [\[CrossRef\]](#)
- Arora, L.; Girija, S.S.; Kapoor, S.; Raj, A.; Pradhan, D.; Shetgaonkar, A. Explainable Artificial Intelligence Techniques for Software Development Lifecycle: A Phase-Specific Survey. In *Proceedings of the IEEE Annual Computers, Software, and Applications Conference (COMPSAC) 2025*, Toronto, ON, Canada, 8–11 July 2025. [\[CrossRef\]](#)

10. Zendeli, B.; Luma, A.; Selimi, B.; Rexha, B. Identifying Limitations in End-to-End Testing: A Systematic Review of Coverage Analysis Techniques. In Proceedings of the 2025 7th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA), Ankara, Türkiye, 23–24 May 2025; pp. 1–9. [\[CrossRef\]](#)
11. Shah, V.; Yadav, P. The Future of Software Testing Automation: Innovations, Challenges, and Emerging Alternatives. In *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*; IEEE: New York, NY, USA, 2025; pp. 1588–1593. [\[CrossRef\]](#)
12. Abhichandani, S.; Vadrevu, N.R.T.; Bagmar, V. AI-Driven Self-Healing in Test Automation: A Review of Autonomous Quality Assurance. In *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*; IEEE: New York, NY, USA, 2025; pp. 1601–1608. Available online: <https://ieeexplore.ieee.org/abstract/document/11069937> (accessed on 1 May 2026). [\[CrossRef\]](#)
13. Trinh, E.; Funke, M.; Lago, P.; Bogner, J. Sustainability Integration of Artificial Intelligence into the Software Development Life Cycle. In Proceedings of the IEEE International Conference on Software Architecture (ICSA) 2024, Hyderabad, India, 4–8 July 2024. [\[CrossRef\]](#)
14. Kim, Y.; Kouatly, R. Self-healing automation testing with Selenium and ChatGPT API integration to improve the efficiency of the maintenance. In Proceedings of the IEEE 2nd International Conference on Frontiers of Artificial Intelligence, Large Language Models and Machine Learning (FLLM), Dubai, United Arab Emirates, 26–29 November 2024. Available online: <https://ieeexplore.ieee.org/document/10852490> (accessed on 25 April 2026).
15. Lops, A.; Narducci, F.; Ragone, A.; Trizio, M. AgoneTest: Automated creation and assessment of Unit tests leveraging Large Language Models. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), Sacramento, CA, USA, 27 October–1 November 2024. [\[CrossRef\]](#)
16. Han, Q.; Shi, Z.; Zhao, Z. Research on trustworthy Software Testing Techniques Based on Large Models. In Proceedings of the IEEE 10th International Symposium on System Security, Safety, and Reliability (ISSSR), Xiamen, China, 30–31 March 2024. Available online: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10562113> (accessed on 1 May 2026).
17. Neelapu, M. *Enhancing Software Testing Efficiency with Generative AI and Large Language Models*; International Journal of Leading Research Publication (IJLRP): Gandhinagar, India, 2024. Available online: <https://www.ijlrp.com/papers/2024/12/1586.pdf> (accessed on 25 April 2026).
18. Sisomboon, W.; Kaewyotha, J.; Songpan, W. Automated Software Test Case Generation Using Directional Partially Weighted Ensemble Large Language Models with Retrieval-Augmented Generation (RAG). *IEEE Access* **2026**, *14*, 31124–31145. [\[CrossRef\]](#)
19. Karpurapu, S.; Myneni, S.; Nettur, U.; Gajja, L.S.; Burke, D.; Stiehm, T.; Payne, J. Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. *IEEE Access* **2024**, *12*, 58715–58721. [\[CrossRef\]](#)
20. Nguyen, D.P.; Maag, S. Codeless web testing using Selenium and machine learning. In Proceedings of the 15th International Conference on Software Technologies (ICSOFT), Paris, France, 7–9 July 2020; pp. 51–60. Available online: <https://hal.science/hal-02909787/> (accessed on 25 April 2026).
21. Pasca, E.M.; Delinschi, D.; Erdei, R.; Matei, O. LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience. *IEEE Access* **2025**, *13*, 56861–56886. [\[CrossRef\]](#)
22. Zhang, Z.; Liu, Y.; Yu, S.; Li, X.; Yun, Y.; Fang, C.; Chen, Z. UniRLTest: Universal Platform-Independent Testing with Reinforcement Learning via Image Understanding. In Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Virtual, 18–22 July 2022. [\[CrossRef\]](#)
23. Komar, M.; Fedorovych, V.; Poidych, V.; Taborovskiy, A. Intelligent System for Visual Testing of Software Products. In Proceedings of the AISD, Munich, Germany, 22–26 July 2024; pp. 9–18. Available online: <https://ceur-ws.org/Vol-3716/paper1.pdf> (accessed on 25 April 2026).
24. Rauf, A.; Alanazi, M.N. Using Artificial Intelligence to Automatically Test GUI. In Proceedings of the IEEE 9th International Conference on Computer Science & Education (ICCSE 2014), Vancouver, BC, Canada, 22–24 August 2014. Available online: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6926420> (accessed on 25 April 2026).
25. Immaculate, S.D.; Begam, M.F.; Floramary, M. Software Bug Prediction Using Supervised Machine Learning Algorithms. In Proceedings of the 2019 International Conference on Data Science and Communication (IconDSC), Bangalore, India, 1–2 March 2019. Available online: <https://ieeexplore.ieee.org/document/8816965> (accessed on 25 April 2026).
26. Xiao, L.; Miao, H.; Shi, T.; Hong, Y. LSTM-based deep learning for spatial-temporal software testing. *Distrib. Parallel Databases* **2020**, *38*, 687–712. [\[CrossRef\]](#)
27. Yang, Y.; Huang, X.; Hao, X.; Liu, Z.; Chen, Z. An Industrial Study of Natural Language Processing Based Test case Prioritization. In Proceedings of the IEEE 10th International Conference on Software Testing, Verification and Validation, Tokyo, Japan, 13–17 March 2017. Available online: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7928016> (accessed on 25 April 2026).

28. Lachmann, R.; Schulze, S.; Nieke, M.; Seidl, C.; Schaefer, I. System-Level Test Case Prioritization Using Machine Learning. In Proceedings of the IEEE 15th International Conference on Machine Learning and Applications, Anaheim, CA, USA, 18–20 December 2016. Available online: <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7838169> (accessed on 25 April 2026).
29. Ali, M.; Mazhar, T.; Arif, Y.; Al-Otaibi, S.; Ghadi, Y.Y.; Shahzad, T.; Khan, M.A.; Hamam, H. Software Defect Prediction Using an Intelligent Ensemble-Based Model. *IEEE Access* **2024**, *12*, 20376–20395. [[CrossRef](#)]
30. Chan, P.Y.P.; Keung, J. Validating Unsupervised Machine Learning Techniques for Software Defect Prediction with Generic Metamorphic Testing. *IEEE Access* **2024**, *12*, 165155–165172. [[CrossRef](#)]
31. Zhang, Y. New Approaches to Automated Software Testing Based on Artificial Intelligence. In Proceedings of the 2024 5th International Conference on Artificial Intelligence and Computer Engineering (ICAICE), Wuhu, China, 8–10 November 2024; pp. 806–810. Available online: <https://ieeexplore.ieee.org/document/10863866> (accessed on 25 April 2026).
32. Nagila, A.; Trivedi, N.; Nagila, R.; Trivedi, K.; Bhardwaj, S.; Rani, J. A Framework for Automated Software Testing Using Machine Learning and Artificial Intelligence. In Proceedings of the 2025 International Conference on Knowledge Engineering and Communication Systems (ICKECS), Chickballapur, India, 23–24 April 2025; pp. 1–7. Available online: <https://ieeexplore.ieee.org/document/11034863> (accessed on 25 April 2026).
33. Muqet, A.; Yue, T.; Ali, S. Mitigating Noise in Quantum Software Testing Using Machine Learning. *IEEE Trans. Softw. Eng.* **2024**, *50*, 2947–2961. [[CrossRef](#)]
34. Guo, Q.; Xie, X.; Li, Y.; Zhang, X.; Liu, Y.; Li, X.; Shen, C. Audex: Automated testing for deep learning frameworks. In Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Virtual, 21–25 December 2020. [[CrossRef](#)]
35. Yahmed, A.H.; Braiek, H.B.; Khomh, F.; Bouzidi, S.; Zaatour, R. DiverGet: A Search-Based Software Testing approach for Deep Neural Network Quantization assessment. *Empir. Softw. Eng.* **2022**, *27*, 193. [[CrossRef](#)]
36. Zolfagharian, A.; Abdellatif, M.; Briand, L.C.; Bagherzadeh, M. A Search-Based Testing Approach for Deep Reinforcement Learning Agents. *IEEE Trans. Softw. Eng.* **2023**, *49*, 3715–3735. [[CrossRef](#)]
37. Gu, Y.; Sun, Y.; Wu, Y.; Yan, F. CNF: An Automated Test Case Generation Method Based on Neuron Coverage Guidance in Key Network Layers. *IEEE Access* **2025**, *13*, 95227–95242. [[CrossRef](#)]
38. Basani, D.K.R.; Kurunthachalam, A. An Intelligent Software Testing Framework For Cloud-Based Robotic Systems Using Ai And Automation. *Int. J. Multidiscip. Eng. Curr. Res.* **2021**, *6*, 31–44.
39. Ho, L.; Esche, M.; Nischwitz, M.; Glesner, S. An Investigation of Machine Learning Solutions to Demonstrate Black Box Software Conformity in Regulated Measuring Instruments. *IEEE Open J. Instrum. Meas.* **2025**, *4*, 1–13. [[CrossRef](#)]
40. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [[CrossRef](#)] [[PubMed](#)]
41. Ansari, A.; Shagufta, M.B.; Fatima, A.S.; Teehreem, S. Constructing Test Cases Using Natural Language Processing. In Proceedings of the IEEE 3rd International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB17), Chennai, India, 27–28 February 2017. Available online: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7972390> (accessed on 25 April 2026).
42. Yang, Z.; Wang, L. IntelliUnitGen: A Unit Test Case Generation Framework Based on the Integration of Static Analysis and Prompt Learning. *IEEE Access* **2025**, *13*, 177760–177784. [[CrossRef](#)]
43. Paduraru, C.; Melemciuc, M. Tool using Machine Learning. In Proceedings of the 13th International Conference on Software Technologies (ICSOFT), Porto, Portugal, 26–28 July 2018. Available online: <https://pdfs.semanticscholar.org/82bd/ef8522578768ffa161a255eb9ce68b99e371.pdf> (accessed on 25 April 2026).
44. Dalal, A.; Tamraparani, V. Self generating & self healing test automation scripts using AI for automating regulatory & compliance functions in financial institutions. *Rev. Intel. Artif. Med.* **2025**, *14*, 784–786.
45. Gupta, A.K.; Govindankutty, S.; Nandyala, A.R.; Thason, J.R.M.; Pippal, S.; Gupta, G.K. AI-Driven Test Automation for Big Data Systems in Cloud Environments. In Proceedings of the IEEE First International Conference on Advances in Computer Science, Electrical, Electronics, and Communication Technologies (CE2CT), Bhimtal, India, 21–22 February 2025. Available online: <https://ieeexplore.ieee.org/abstract/document/10941644> (accessed on 25 April 2026).
46. Caglar, O. Development of Cloud and Artificial Intelligence based Software Testing Platform (ChArIoT). In Proceedings of the 2023 Innovations in Intelligent Systems and Applications Conference (ASYU), Sivas, Turkey, 11–13 October 2023. Available online: <https://ieeexplore.ieee.org/document/10296551> (accessed on 25 April 2026).
47. Loubiri, O.; Maag, S. Automated Web Testing using Machine Learning and Containerization. In Proceedings of the IEEE 26th International Conference on Circuits, Systems, Communications and Computers (CSCC), Rhodes Island, Greece, 19–22 July 2022. Available online: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10017868> (accessed on 25 April 2026).

48. Khankhoje, R. AI-Based Test Automation for Intelligent Chatbot Systems. *Int. J. Sci. Res. (IJSR)* **2023**, *12*, 1302–1309. [[CrossRef](#)]
49. Wan, C.; Liu, S.; Xie, S.; Liu, Y.; Hoffmann, H.; Maire, M.; Lu, S. Automated testing of software that uses machine learning APIs. In Proceedings of the 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 22–27 May 2022. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.